

01 · Speicherphysik & RAM-Typen

Mikrocomputertechnik 3 · DHBW Stuttgart

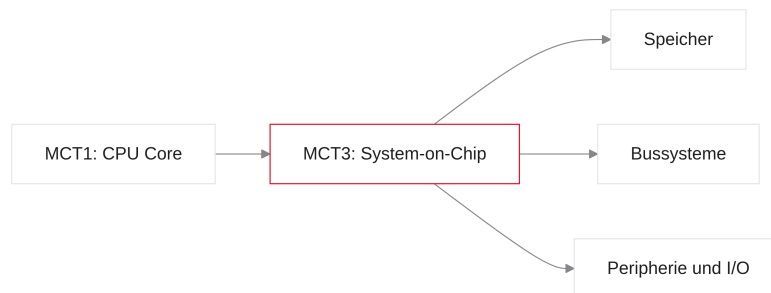
Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 88

Willkommen zu Mikrocomputertechnik 3!

- Der Kreis schließt sich: dritter und finaler Teil der Mikrocomputertechnik-Reihe
- Rückblick MCT1: Prozessor auf Gatter-Ebene aus Nullen und Einsen aufgebaut
- Nächster Schritt: von der isolierten CPU zum Gesamtsystem
- System-on-Chip (SoC): CPU, Speicher, Busse und Peripherie im Zusammenspiel
- Hardware trifft Software: von nacktem Assembler zur hardwarenahen C-Programmierung



Notizen

Herzlich willkommen zu Mikrocomputertechnik 3! Es freut mich sehr, Sie nach MCT1 wiederzusehen. Sie haben bereits ein enormes Verständnis dafür entwickelt, wie ein Prozessor intern tickt — von der ALU bis zum Pipelining. Jetzt zoomen wir heraus. Ein Prozessor allein kann nichts bewirken, wenn er keine Daten speichern oder mit der Außenwelt kommunizieren kann. In diesem Semester bauen wir gedanklich ein komplettes System-on-Chip (SoC) um unseren RISC-V-Kern herum. Wir werden C-Code schreiben, der direkt mit der Hardware spricht.

Rückblick: Was wir aus MCT1 wissen

- Die RISC-V ISA: eleganter, modularer Befehlssatz (RV32I)
- Die Register-Werkbank: 32 Architektur-Register als schneller, aber winziger Arbeitsbereich
- Der Datenpfad: Fetch, Decode, Execute, Memory und Write-Back
- Pipelining: mehrere Befehle gleichzeitig auf dem Fließband
- Memory-Mapped I/O: Geräte tarnen sich als Adressen im Arbeitsspeicher

```
# MCT1 Refresher: Basisadresse und I/O-Register
li    s1, 0xF0000000
lw    t0, 0(s1)
andi  t1, t0, 1
beq   t1, zero, Wait
```

Notizen

Erinnern wir uns kurz an unser Handwerkszeug aus MCT1. Wir kennen die Architektur, wir wissen, was ein Load- und ein Store-Befehl auf dem Silizium auslösen und wie Pipelining den Durchsatz maximiert. Am Ende von MCT1 haben wir bereits gesehen, dass LEDs und Schalter über Memory-Mapped I/O angesteuert werden können. Genau an dieser Stelle setzen wir an: Wenn alles über Speicheradressen läuft, müssen wir verstehen, wie diese Speicher physikalisch aufgebaut sind.

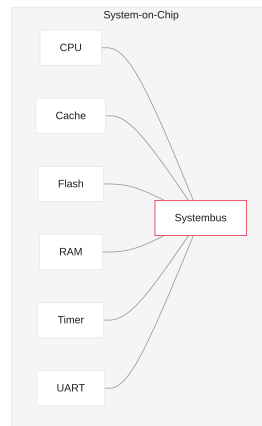
Der Paradigmenwechsel: Core vs. SoC

- Der Core (CPU): reines Rechenwerk (ALU, Register, Pipeline)
- Früher: CPU, RAM und I/O als getrennte Bausteine auf der Platine
- Heute: System-on-Chip — alles auf einem Silizium-Die
- Vorteile: weniger Strom, Kosten und Platz; Herausforderung: Systembus

Notizen

Der Begriff Mikroprozessor ist heute eigentlich fast überholt; wir sprechen meistens von Mikrocontrollern oder System-on-Chips. Warum? Weil es aus physikalischen und finanziellen Gründen viel sinnvoller ist, den Arbeitsspeicher, den Festspeicher und die Peripherie-Controller direkt mit der CPU auf dasselbe Stück Silizium zu drucken. Das spart Latenz und Energie, da keine Signale über kupferne Leiterbahnen einer Platine geschickt werden müssen.

SoC: Komponenten am Systembus



Notizen

Im SoC hängen CPU, Cache, Flash, RAM und Peripherie am gemeinsamen Systembus. Der Bus muss den Datenhunger der CPU stillen — ohne ihn bleibt selbst der schnellste Kern untätig.

Unsere Werkzeuge in MCT3

- Hochsprache statt Assembler: Programmierung in C (Embedded Systems)
- Toolchain: RISC-V GCC übersetzt C in Maschinencode
- Simulator: von Venus zu Renode
- Fokus: Hardware-Treiber statt Sortier-Algorithmen
- Bare-Metal: direkt auf der Hardware, ohne Betriebssystem



Notizen

Da wir nun komplexe Systeme bauen, reicht uns nackter Assembler nicht mehr — die Produktivität wäre zu gering. Wir nutzen C. C ist der Industrie-Standard für hardwarenahe Programmierung, da es uns erlaubt, Adressen als Pointer direkt anzufassen, während uns der Compiler die mühsame Register-Zuweisung abnimmt. Als Laborumgebung nutzen wir Renode, einen professionellen Open-Source-Simulator, der komplette SoCs simulieren kann.

Was ist Renode?

- Industrie-Standard: u. a. bei Google, Intel und Microchip zur Chip-Simulation
- Vollständige SoC-Simulation: CPU, Busse, Timer und RAM
- Professionelles Kommandozeilen-Tool (ohne grafisches Breadboard)
- Virtuelle I/O: Sensoren per Skript an I2C- oder SPI-Bus
- Skript-gesteuert: SoCs über Textdateien (.resc / .repl)

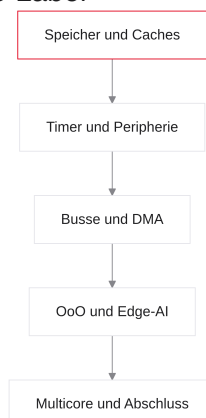
```
# Beispielhaftes Renode Setup-Skript
machine LoadPlatformDescription @platforms/cpus/riscv32.repl
sysbus LoadELF @firmware.elf
start
```

Notizen

Renode ist ein fantastisches Werkzeug. Anstatt zu warten, bis eine Chipfabrik das neue Silizium ausliefert, können Software-Ingenieure in Renode bereits den Code für den neuen Chip schreiben. Renode simuliert das exakte Timing und die Memory Map des Chips. Sie werden im Labor lernen, wie man C-Code lokal kompiliert und das fertige Binary direkt in diese simulierte Hardware lädt.

Modul-Übersicht: Unser Fahrplan

- Block 1 & 2: Speicherphysik, RAM-Typen, Caches und die Memory Wall
- Block 3 & 4: Interrupts, Hardware-Timer, PWM und Watchdogs
- Block 5 & 6: Serielle Busse (UART, I2C, SPI) und DMA
- Block 7 & 8: Out-of-Order Execution und Edge-AI (KI-Beschleuniger)
- Block 9: Multicore und Abschluss-Labor



Notizen

Unser Semester ist logisch gegliedert. Wir starten ganz innen im Chip bei den Speichern und Caches. Dann arbeiten wir uns langsam nach außen: über die System-Timer hin zu den externen Bussen, mit denen wir Sensoren auslesen. In der letzten Phase des Semesters widmen wir uns den hochmodernen Konzepten: Out-of-Order CPUs und maschinelles Lernen direkt auf dem Mikrocontroller (Edge AI).

Folie

8 / 88

Prüfungsleistung und Laborbetrieb

- Theorie und Praxis: je Einheit 2 Stunden Theorie und 2 Stunden Labor
- Labor als Schlüssel: C-Treiber für das Verständnis der SoC-Architektur
- Prüfungsform: schriftliche Klausur am Semesterende (ca. 90 Minuten)
- Klausurinhalte: C-Code, Pointer-Arithmetik, Architektur, Cache-Berechnungen
- Fokus auf Problemlösung, nicht auf Auswendiglernen von Datenblättern

Notizen

Wie in MCT1 gilt auch hier: Die Vorlesung gibt Ihnen die Konzepte, aber das echte Verständnis kommt im Labor. Wer die Laborblätter selbstständig in Renode löst, hat bereits 80 % der Klausurvorbereitung erledigt. In der Klausur werden wir keine Registeradressen eines bestimmten Chips abfragen, sondern Konzepte: Wie richten Sie einen DMA-Transfer ein? Warum tritt ein Cache-Miss auf?

Folie

9 / 88

Agenda für den heutigen Tag

1. Systemarchitektur (Von-Neumann vs. Harvard) und Memory Wall
2. Speicher-Klassifikationen (volatil, non-volatil, Random Access)
3. Festwertspeicher (ROM, EPROM, Flash) — Architektur und Nutzung
4. Volatiler Speicher (SRAM vs. DRAM) — Transistoren vs. Kondensatoren
5. Spezialspeicher im SoC (DPRAM, FIFOs) und Brückenschlag zum Labor

Notizen

Unser heutiges Ziel ist es, das Konzept „Arbeitsspeicher“, das wir in MCT1 als magische Blackbox betrachtet haben, physikalisch zu öffnen. Wir klären heute ein für alle Mal den Unterschied zwischen dem Speicher, auf dem unser Code liegt, und dem Speicher, in dem unsere Variablen leben.

Lernziele Block 1

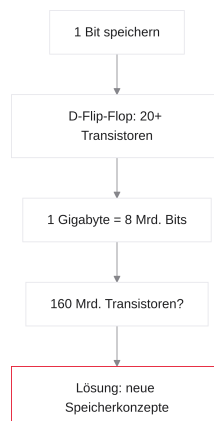
- Memory Wall und Speicherhierarchie-Idee motivieren
- Volatil vs. non-volatil sowie Random Access vs. sequentiell einordnen
- NOR-Flash/XIP vs. NAND und Wear Leveling erklären
- SRAM (6T) vs. DRAM (1T1C, Refresh) unterscheiden
- Memory Map und Linker-Sections (`.text`, `.data`, `.bss`) zuordnen

Notizen

Harris & Harris, *Digital Design and Computer Architecture — RISC-V Edition*, Kap. Memory Systems — Terminologie und Aufbau.

Die Leitfrage des Tages

- Die CPU ist schnell: Milliarden Berechnungen pro Sekunde
- Datenhunger: jede Berechnung braucht Befehle und Daten
- Masse: moderne Programme bestehen aus Milliarden von Bits
- Flip-Flop-Problem: Milliarden D-Flip-Flops sind zu groß, teuer und stromhungrig
- Leitfrage: Wie speichert Silizium Milliarden Bits dicht und effizient?

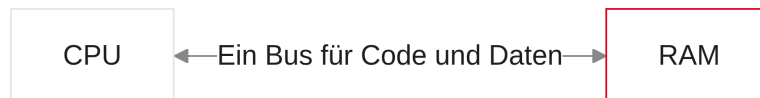


Notizen

Das D-Flip-Flop aus MCT1 ist großartig, um 32 Register zu bauen. Aber wenn wir ein Gigabyte Speicher bauen wollen und für jedes Bit 20 Transistoren brauchen, würde unser Chip so groß wie ein Fußballfeld werden und sofort verglühen. Wir brauchen grundlegend andere physikalische Konzepte, um Daten dicht und effizient zu speichern. Das ist die Essenz der heutigen Vorlesung.

Systemarchitektur: Von-Neumann

- Klassisches Modell nach John von Neumann (1945)
- Gemeinsamer Speicher: Befehle und Daten im selben Arbeitsspeicher
- Gemeinsamer Bus für Adressen und Daten zwischen CPU und RAM
- Von-Neumann-Flaschenhals: kein paralleles Laden von Befehl und Variable
- Folge: Structural Hazards auf dem Fließband



Notizen

Starten wir mit der Systemarchitektur. Die Von-Neumann-Architektur ist das Konzept, das fast alle modernen Desktop-PCs nutzen. Alles liegt im selben RAM. Der große Nachteil: Wenn der Datenpfad aus MCT1 einen Load-Befehl ausführt, muss er Daten aus dem RAM holen. Wenn er gleichzeitig den nächsten Befehl fetchen möchte, knallt es auf dem Bus. Dieser Stau ist der berühmte Von-Neumann-Flaschenhals.

Systemarchitektur: Harvard

- Harvard-Architektur trennt die Speicherräume strikt
- Getrennte Speicher für Befehle (Code) und Daten
- Getrennte Bus-Systeme zur CPU
- Vorteil: Fetch und Memory im selben Taktzyklus möglich
- Modified Harvard: extern Von-Neumann, intern getrennte L1-Caches



Notizen

In der Harvard-Architektur entzerren wir den Verkehr. Wir haben zwei Busse. Genau dieses Modell haben wir in MCT1 für unseren 5-Stufen-Prozessor genutzt: einen Instruction Memory und einen Data Memory. In echten SoCs findet man heute oft das „Modified Harvard“-Design: Der Hauptspeicher draußen ist ein großer Block (Von-Neumann), aber direkt am Prozessorkern gibt es getrennte L1-Instruction- und L1-Data-Caches, die wie eine Harvard-Architektur wirken.

Die Memory Wall

- Moores Gesetz: Transistoren kleiner, CPUs schneller
- Ungleichgewicht: RAM-Geschwindigkeit hielt nicht mit
- Memory Wall: wachsende Lücke zwischen CPU- und RAM-Speed
- Warte-Problem: oft mehr Stalls als echte Rechenzeit
- Lösung: hierarchischer Aufbau in mehreren Speicherschichten

Generation	CPU relativ	RAM relativ
früh	1×	1×
später	16×	≈ 3×

Schematisch — Trendillustration, keine Messdaten.

Notizen

Die Tabelle ist bewusst schematisch — sie illustriert den Trend, keine konkreten Messreihen. Jahrelang haben wir CPUs höher getaktet. Aber der Arbeitsspeicher, der auf einer anderen Platine liegt, wird physikalisch nicht im selben Maß schneller. Die Folge: Die CPU verhungert am Datenbus. Wenn eine CPU bei 4 GHz arbeitet und der RAM 100 Nanosekunden für eine Antwort braucht, vergehen 400 Taktzyklen, in denen die CPU einfach nichts tun kann!

Das magische Speicher-Dreieck

Drei Ziele, die sich gegenseitig ausschließen:

1. Geschwindigkeit — wie schnell lesen/schreiben?
2. Kapazität — wie viele Gigabyte auf dem Chip?
3. Kosten — wie teuer ist ein Bit in der Herstellung?

Eiserne Regel: immer nur zwei Ziele gleichzeitig optimierbar.

Beispiel: schnell und groß ist extrem teuer (SRAM); groß und billig ist langsam (Massenspeicher).



Notizen

Dies ist das magische Dreieck der Speicherarchitektur. Wenn jemand zu Ihnen kommt und einen Speicher fordert, der rasend schnell ist, Terabytes an Daten fasst und nur 5 Euro kostet, müssen Sie ihn enttäuschen. Die Physik lässt das nicht zu. Deshalb bestehen Computer nicht aus einem einzigen Speichertyp, sondern aus einer Mischung verschiedener Technologien.

Folie

16 / 88

Klassifikation 1: Volatil vs. Non-Volatil

Volatiler Speicher (flüchtig)

- Verliert Daten sofort ohne Betriebsspannung
- Beispiele: SRAM, DRAM, CPU-Register
- Vorteil: meist schnell und beliebig oft beschreibbar

Non-volatiler Speicher (nicht-flüchtig)

- Behält Daten Jahre ohne Strom
- Beispiele: Flash, SD-Karten, ROM
- Nachteil: oft langsamer beim Schreiben, physischer Verschleiß

Notizen

Die grundlegendste Unterscheidung von Speichern ist die Frage nach dem Stromausfall. Volatile Speicher sind das Kurzzeitgedächtnis des Computers. Ziehen Sie den Stecker, ist alles weg. Non-Volatile Speicher sind das Langzeitgedächtnis. Für ein funktionierendes System (wie unseren SoC) brauchen wir zwingend beides: einen Festspeicher für Firmware bzw. Boot-Code und einen Arbeitsspeicher für die laufenden Berechnungen.

Klassifikation 2: Random Access vs. sequenziell

Random Access (wahlfreier Zugriff)

- Jede Adresse in derselben Zeit direkt auslesbar
- Latenz unabhängig vom Ort (Beispiel: RAM)

Sequenzieller Zugriff

- Daten nur der Reihe nach (wie eine Kassette)
- Ende erreicht man erst nach dem Durchlaufen der davorliegenden Daten

Architektur-Bedeutung: CPUs führen Code nur aus Random-Access-Speichern direkt aus.

Notizen

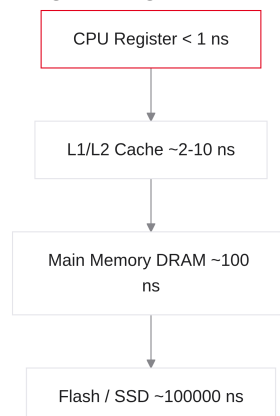
Der Begriff „RAM“ (Random Access Memory) wird oft synonym für Arbeitsspeicher genutzt, aber eigentlich beschreibt er eine Zugriffsmethode. „Wahlfrei“ bedeutet, die Latenz ist unabhängig vom Ort der Daten. Wenn Sie ein Magnetband haben, dauert es Minuten, um an das Ende des Bandes zu spulen. Eine CPU braucht aber für jeden Befehl im Speicher sofortigen Zugriff, daher muss Code zwingend in einem Random-Access-Speicher liegen.

Die Speicherhierarchie-Pyramide

Um die Memory Wall zu überwinden, stapeln Architekten Speicher übereinander:

- Spitze: CPU-Register (klein, teuer, blitzschnell)
- Ebene 2: Caches L1/L2/L3 (SRAM — Megabytes, sehr schnell)
- Ebene 3: Main Memory (DRAM — Gigabytes, moderat)
- Basis: Massenspeicher (Flash/SSD — Terabytes, langsam, billig, non-volatil)

Prinzip: Daten wandern je nach Nutzungshäufigkeit in der Pyramide.



Notizen

Da wir das magische Dreieck nicht brechen können, bauen wir eine Pyramide. Oben haben wir extrem teuren, rasend schnellen Speicher, aber eben nur sehr wenig davon. Unten haben wir den billigen, riesigen Speicher. Die Kunst der Computerarchitektur — und das werden wir in Block 2 ausführlich beleuchten — liegt darin, die Daten, die die CPU in den nächsten Millisekunden braucht, unbemerkt in der Pyramide nach oben zu schieben.

Folie

19 / 88

Fokus heute: Wir öffnen die Chips

- Heute: Physik statt Betriebssystem
- Wir öffnen die Boxen der Speicherhierarchie
- Wie funktioniert Festwertspeicher (ROM/Flash)?
- Warum nutzen SRAM und DRAM andere physikalische Konzepte?
- Was bedeutet das für uns als C-Programmierer am SoC?

Notizen

Nachdem wir nun die Motivation und die Einordnung im System haben, greifen wir uns heute zwei fundamentale Speicherarten heraus: den Festspeicher, auf dem unsere Firmware liegt, und den volatilen Arbeitsspeicher. Wir werden sehen, dass beide extrem unterschiedliche Kompromisse beim Design eingehen mussten.

Folie

20 / 88

Festwertspeicher (Non-Volatile)

- Start unten in der Pyramide: Speicher, der Stromausfall überlebt
- Überbegriff: ROM (Read-Only Memory)
- Historisch wirklich nur lesbar
- Evolution: Firmware nachträglich updatebar machen
- Weg: Mask-ROM → PROM → EPROM → EEPROM → Flash

Notizen

Beginnen wir mit dem Langzeitgedächtnis. Jeder Computer muss beim Einschalten wissen, was er tun soll. Er braucht Boot-Code. Dieser Code muss zwingend auf einem nicht-flüchtigen Speicher liegen. Die Geschichte dieses Speichers ist eine faszinierende Reise davon, wie Hardware immer flexibler und softwareähnlicher wurde.

Mask-ROM

- Nullen und Einsen bei der Herstellung physisch verdrahtet (Fotolithografie-Masken)
- Vorteil: extrem billig in der Massenproduktion
- Nachteil: Bug $\Rightarrow$ alle Chips unbrauchbar
- Früher: Game-Cartridges, Spieluhren
- Heute: kaum noch für Anwendercode; ggf. unveränderliche Boot-Loader

Notizen

Der Mask-ROM ist Hard-Coded im wahrsten Sinne des Wortes. Eine Eins ist eine physische Kupferverbindung, eine Null ist eine durchtrennte Leitung. Wenn Sie als Programmierer einen Fehler gemacht haben, mussten Sie den Halbleiter-Hersteller anrufen, eine neue Belichtungsmaske für 100.000 Dollar bestellen und alle produzierten Chips entsorgen. Die Industrie verlangte dringend nach einer besseren Lösung.

PROM (Programmable ROM)

- Chip verlässt das Werk leer (alles Einsen) und wird beim Kunden beschrieben
- Jede Zelle besitzt eine mikroskopische Sicherung (Fuse)
- Hohe Spannung „brennt“ die Sicherung durch: $1 \rightarrow 0$
- OTP (One-Time Programmable): einmal durchgebrannt, nie reparierbar
- Gut für kleine Serien; Firmware-Updates weiterhin unmöglich

Notizen

Das PROM war der erste Schritt zur Flexibilität. Man kauft leere Chips und brennt sein Programm mit einem speziellen Programmiergerät hinein. Das passiert wörtlich: Man jagt eine hohe Spannung durch den Chip und brennt winzige Sicherungen durch. Das ist großartig für Kleinserien, aber „One-Time Programmable“ heißt: Bei einem Bug müssen Sie zwar nicht die Maske ändern, aber den Chip müssen Sie trotzdem auslöten und wegwerfen.

EPROM (Erasable PROM)

- Speicher, der gelöscht und neu beschrieben werden kann
- Quarzglas-Fenster auf der Chip-Oberseite
- UV-Licht (ca. 20–30 Minuten) löscht den gesamten Chip
- Nachteil: immer ganzer Chip, langsam, Ausbau nötig
- Fenster nach dem Brennen abdecken, sonst löscht Sonnenlicht den Code

Notizen

Wenn Sie heute alte Arcade-Automaten oder Synthesizer aus den 80er Jahren aufschrauben, sehen Sie diese wunderschönen Keramikchips mit einem runden Glasfenster in der Mitte. Das ist ein EPROM. Die Idee war revolutionär: Man konnte den Chip unter eine UV-Lampe legen, ihn komplett löschen und neu beschreiben. Es dauerte eine halbe Stunde, aber es sparte enorm viel Geld bei der Entwicklung.

EEPROM (Electrically Erasable PROM)

- Löschen per elektrischem Signal statt UV-Licht
- In-System Programming: kein Auslöten nötig
- Byte-Ebene: einzelne Bytes gezielt löschen und neu beschreiben
- Kehrseite: große Zellen, geringe Dichte, hohe Kosten pro Megabyte
- Heute: kleine Konfigurationsdaten (MAC-Adresse, Kalibrierung)

Notizen

Das UV-Licht war unpraktisch. Das EEPROM löste dieses Problem, indem man durch geschicktes Anlegen von hohen elektrischen Spannungen die Elektronen wieder aus der Speicherzelle herausziehen konnte. Das Geniale: Man konnte gezielt ein einzelnes Byte löschen, während der Rest des Chips unangetastet blieb. Der Nachteil war jedoch die enorme Größe der einzelnen Zelle, weshalb EEPROMs niemals Gigabyte-Größen erreichten.

Flash-Speicher: Die Revolution der Blöcke

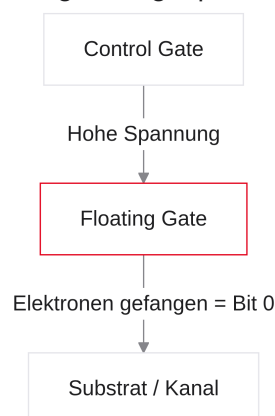
- Geburt des Flash (Toshiba, 1984): Kompromiss aus EPROM und EEPROM
- Löschen nicht byteweise, sondern blockweise „blitzartig“
- Architektur-Trick: Wegfall der byteweisen Löschlogik $\Rightarrow$ kleinere Zellen
- Resultat: hohe Speicherdichte zu niedrigen Preisen
- Dominante Technologie für Firmware und Massenspeicher (SSD, USB, Smartphones)

Notizen

In den 80er Jahren suchte die Industrie nach einem Weg, die Vorteile von EEPROMs (elektrisches Löschen) mit einer hohen Speicherdichte zu vereinen. Der Ingenieur Fujio Masuoka bei Toshiba erfand den Flash-Speicher. Sein Trick: Er warf die komplexe Verschaltung weg, die nötig war, um jedes Byte einzeln zu löschen. Flash kann nur blockweise gelöscht werden. Das sparte so viel Platz auf dem Silizium, dass Flash die Welt eroberte.

Das Floating-Gate (logische Sicht)

- Flash speichert Bits ohne Strom über Floating-Gate-Transistoren
- Zwei Gates: Control-Gate und isoliertes Floating-Gate
- Floating-Gate vollständig in Siliziumdioxid eingeschlossen
- Hohe Spannung ($\approx +20\text{ V}$): Elektronen tunneln ins Floating-Gate
- Ohne Spannung: Elektronen gefangen — gespeichertes Bit (oft Jahre haltbar)



Notizen

Wir lassen die Quantenphysik außen vor und betrachten es logisch: Ein normaler Transistor schaltet Strom. Beim Flash bauen wir eine kleine, isolierte Insel in den Transistor ein — das Floating-Gate. Wenn wir sehr viel Spannung anlegen, tunneln Elektronen durch die Isolationsschicht auf diese Insel. Wenn wir den Strom abschalten, können sie nicht mehr zurück. Sie sind gefangen. Diese gefangene Ladung

verändert das Schaltverhalten des Transistors. Die CPU misst diesen Transistor und weiß: Hier ist eine Null gespeichert.

Folie

27 / 88

NAND vs. NOR Flash

Flash spaltet sich in zwei Architekturen — je nach Verdrahtung der Floating-Gate-Zellen:

	NOR	NAND
Verdrahtung	parallel	seriell (Kette)
Zugriff	Byte/Wort (Random Access)	Seiten (Pages)
Typischer Einsatz	Mikrocontroller	SSD, Smartphones

Notizen

Flash ist nicht gleich Flash. Je nachdem, wie Sie diese isolierten Transistoren auf dem Silizium anordnen, erhalten Sie völlig unterschiedliche Eigenschaften. NOR-Flash verdrahtet die Zellen parallel, was viel Platz kostet, aber extrem schnellen und direkten Zugriff erlaubt. NAND-Flash schaltet sie in Serie, was Platz spart, den Zugriff aber sehr starr und blockbasiert macht.

Folie

28 / 88

NOR-Flash & Execute-In-Place (XIP)

- Random Access: direkte Adressierung, z. B. `0x08000004`
- XIP: CPU holt Befehle direkt aus dem NOR-Flash (ohne RAM-Kopie)
- Fetch-Stage der RISC-V-Pipeline liest Code aus dem Flash
- Nachteil: teurer, geringe Kapazität (oft 1–16 MB auf dem SoC)
- In MCT3: `.text`-Section landet im NOR-Flash des Controllers

Notizen

NOR-Flash verhält sich beim Lesen genau wie das SRAM aus unserer Architektur-Vorlesung. Das ist essenziell für kleine Embedded-Systeme! Die CPU schickt eine Adresse, der NOR-Flash antwortet sofort mit dem Assembler-Befehl. Das nennt man Execute-In-Place (XIP). Das Programm läuft direkt aus dem Flash. Das spart immens viel teuren Arbeitsspeicher, da der Code nicht erst kopiert werden muss.

C-Code und der Flash-Speicher

- `const` zwingt den Compiler zu ROM/Flash (Section `.rodata`), nicht RAM
- C-Pointer können direkt auf Flash-Adressen zeigen
- Lesen erlaubt: `int wert = *flash_ptr;` (schnell, XIP)
- Schreiben verboten: `*flash_ptr = 42;` → Store/AMO Access Fault (mcause 7; kein einfacher Store)

```
// Pointer auf internen NOR-Flash
const uint32_t *firmware_version =
    (const uint32_t *) 0x08000000;

// Lesen klappt sofort (Random Access)
uint32_t ver = *firmware_version;
```

Notizen

Hier schlagen wir die Brücke zur C-Programmierung. Wenn Sie große Tabellen, Bilder oder Fonts in Ihrem Programm haben, setzen Sie das Keyword `const` davor. Der Linker legt diese Daten in den Flash. Da NOR-Flash Random-Access bietet, können Sie per Pointer darauf zugreifen, als wäre es normaler RAM. Aber wehe, Sie versuchen per Software einen Wert dorthin zu speichern — das wird von der Hardware blockiert, da Flash-Speicher einen aufwendigen Erase-Write-Zyklus benötigt.

NAND-Flash (Massenspeicher)

- Höchste Dichte durch serielle Verkettung der Zellen
- Kein Random Access: immer ganze Page (z. B. 4096 Bytes) lesen
- Kein XIP: CPU führt Code nicht direkt aus NAND aus
- Boot: Code zuerst von NAND nach DDR-RAM kopieren
- Einsatz: SSDs, SD-Karten, USB-Sticks (Terabyte-Bereich)

Notizen

NAND-Flash hat die Datenspeicherung revolutioniert. Es ist der Grund, warum wir Terabytes in unseren Laptops haben. Aber die CPU hasst NAND-Flash. Wenn der Program Counter einen Befehl holen will, kann der NAND-Flash nicht antworten, sondern liefert nur einen riesigen Datenblock zurück. Deshalb muss beim Start Ihres Laptops das Betriebssystem erst langwierig von der SSD in den Arbeitsspeicher (RAM) kopiert werden.

Die Flash-Krankheit: Wear and Tear

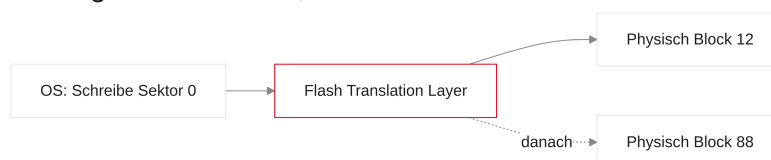
- Geheimnis: Flash verschleißt bei jedem Schreibvorgang
- Hohe Spannung beim Tunneln beschädigt die Oxid-Isolation
- Limit: ca. 10 000 bis 100 000 Erase/Write-Zyklen — je nach Zelltyp (NOR/SLC höher, TLC/QLC oft nur ~ 1000 – 10000)
- Danach: Zelle hält Ladung nicht mehr (physisch defekt)
- Gefahr: ständiges Schreiben in denselben Sektor zerstört den Chip

Notizen

Hier müssen Sie als Systemarchitekt extrem aufpassen. Wenn Sie RAM beschreiben, können Sie das unendlich oft tun. Wenn Sie Daten in einen Flash-Speicher schreiben, beschädigen Sie ihn auf mikroskopischer Ebene. Die Glasisolierung bekommt Risse, durch die die gefangenen Elektronen wieder entkommen. Nach 100.000 Schreibvorgängen ist der Sektor tot.

Die Rettung: Wear Leveling (FTL)

- Wear Leveling: Verschleißausgleich über wechselnde physische Blöcke
- Logischer Sektor → jedes Mal anderer physischer Flash-Block
- Flash Translation Layer (FTL) übersetzt logisch $\leftrightarrow$ physisch
- SD-Karte/SSD: eigener Controller; Bare-Metal: Software muss Wear Leveling leisten



Notizen

Damit Ihre teure SSD nicht nach einem Jahr defekt ist, trickst der Controller. Wenn das Betriebssystem sagt „Schreibe Sektor 0“, schreibt die SSD es in Wirklichkeit in Block 12. Beim nächsten Mal in Block 88. Ein interner Algorithmus (Wear Leveling) verteilt die Schreibzugriffe gleichmäßig über alle Millionen Zellen des Chips, damit alle gleichzeitig altern. Wenn Sie in MCT3 auf einem nackten RISC-V SoC Daten in den Flash schreiben wollen, haben Sie diese Luxus-Schicht nicht. Sie müssen das selbst programmieren!

Memory Map: Wo liegt der Flash im SoC?

- In der 32-Bit Memory Map ist ein Block für Flash reserviert
- Beispiel (STM32-artig): `0x0800_0000` bis `0x080F_FFFF` (1 MB NOR-Flash)
- RISC-V/SiFive oft anders: Boot-ROM bei `0x0000_1000`, QSPI-Flash ab `0x2000_0000`
- Linker legt Maschinen-Code (`.text`) auf die Plattform-Startadresse
- Nach Reset: PC springt hartverdrahtet auf den Reset-Vektor (plattformspezifisch)
- Boot per Execute-in-Place aus dem NOR-Flash

Notizen

Lassen Sie uns den Bogen zurück zu MCT1 schlagen. Der Prozessor legt eine 32-Bit Adresse auf den Bus. Der Adress-Decoder erkennt den Flash-Bereich, weckt den NOR-Flash auf und leitet den Befehl an die CPU. Die konkreten Basisadressen sind plattformspezifisch — STM32 nutzt oft `0x0800_0000`, SiFive/Renode-RISC-V andere Maps. Das Prinzip bleibt: Boot per XIP aus NOR-Flash.

Zusammenfassung: Non-Volatile Memory

- Festwertspeicher überlebt Stromausfall und hält Firmware/Boot-Code
- Evolution: Mask-ROM bis blockbasiertes Flash
- NOR-Flash: parallel, XIP, Standard in Mikrocontrollern
- NAND-Flash: seriell, dicht, Massenspeicher; braucht RAM zum Ausführen
- Flash verschleißt (Oxid-Schäden) und braucht Wear Leveling

Notizen

Damit haben wir die Basis unserer Speicherpyramide durchleuchtet. Wir wissen nun, wo unser Code liegt, wenn der Strom ausgeschaltet ist, und warum der Prozessor ihn direkt ausführen kann. Aber der Flash-Speicher hat ein Problem: Er ist viel zu langsam für die Arbeitsvariablen unserer Software und er verschleißt, wenn wir ihn ständig mit neuen Werten beschreiben würden.

Der Bedarf an Arbeitsspeicher

Wir brauchen Speicher, der:

- im Takt der CPU (Nanosekunden) liest und schreibt
- unendlich oft beschreibbar ist, ohne zu verschleißen

Antwort: volatiler Speicher (RAM).

Zwei Technologien: **SRAM** und **DRAM**.

Notizen

Für Variablen, Zwischenergebnisse der ALU und den Stack brauchen wir ein perfektes Kurzzeitgedächtnis. Als Nächstes öffnen wir den volatilen Arbeitsspeicher: zuerst SRAM, dann DRAM.

Fragen? — Non-Volatile

- Flash-Architektur?
- Execute-in-Place (XIP)?
- Wear Leveling?

Notizen

Bevor wir in die Transistorebene von SRAM und DRAM eintauchen: Ist klar, warum Mikrocontroller Code typischerweise aus NOR-Flash (XIP) ausführen und nicht aus NAND?

Kurze Pause

10 Minuten Pause.

Danach:

- SRAM (6-Transistor-Zelle)
- DRAM (1-Transistor-Zelle und Refresh)
- Spezialepeicher (FIFOs und Dual-Ported RAM)

Notizen

Atmen Sie kurz durch. Nach der Pause folgt die Physik des Arbeitsspeichers.

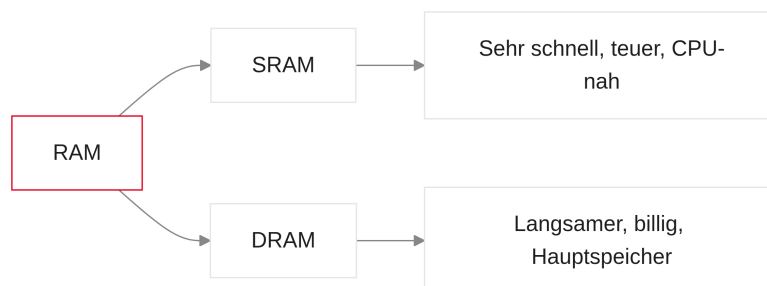
Volatiler Speicher: RAM

- Arbeitsspeicher: Kurzzeitgedächtnis des Prozessors
- Volatil: Daten weg ohne Versorgungsspannung
- Wahlfreier Zugriff: jede Adresse gleich schnell
- Zwei Architekturen: SRAM (Static RAM) und DRAM (Dynamic RAM)
- Gleicher Zweck, grundlegend andere Physik

Notizen

Nachdem wir geklärt haben, wie Code den Stromausfall überlebt, widmen wir uns den flüchtigen Speichern. Wenn Sie in C ein Array anlegen oder eine Variable definieren, leben diese im RAM. Damit der Prozessor nicht unnötig wartet, muss dieser Speicher schnell sein. Wir unterscheiden statischen und dynamischen RAM.

RAM: SRAM und DRAM



Notizen

Schematisch: SRAM sitzt nahe an der CPU (Caches, Register), DRAM bildet den großen Hauptspeicher. Beides speichert Bits — mit völlig unterschiedlichen Bauteilen.

SRAM — Prinzip

- Statisch: Wert bleibt ohne Auffrischen, solange Strom anliegt
- Kein Refresh nötig (im Gegensatz zu DRAM)
- Schnellste verfügbare Speichertechnologie
- Latenz oft unter 1 ns
- Kehrseite: geringe Dichte — viele Transistoren pro Zelle

Notizen

„Statisch“ bedeutet hier nicht, dass Daten bei Stromausfall erhalten bleiben, sondern dass das Bit im Transistornetzwerk stabil liegt, solange die Betriebsspannung da ist. SRAM ist das Rückgrat schneller Caches und Register.

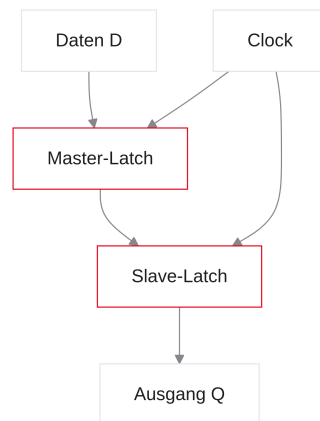
Refresher: Das D-Flip-Flop (MCT1)

- Register in MCT1: D-Flip-Flops
- Master-Slave-D-FF: oft über 20 Transistoren
- 1 MB mit Flip-Flops: Hunderte Millionen Transistoren
- Zu viel Fläche und Abwärme
- Lösung: Design auf das physikalische Minimum eindampfen

Notizen

Für 32 Register sind Flip-Flops vertretbar. Für Megabyte-Caches im Die brauchen wir eine radikal kleinere Zelle.

D-Flip-Flop: Master und Slave



Notizen

Schematisch: Zwei Latches hintereinander. Funktional stark — flächenmäßig zu teuer für großen SRAM.

Folie

43 / 88

Die 6T-SRAM-Zelle

- Klassisches SRAM-Bit: exakt 6 Transistoren (6T)
- Vier Transistoren: zwei kreuzgekoppelte Inverter
- Zwei Access-Transistoren als „Türsteher“
- Wordline (WL): öffnet die Zelle
- Bitlines (BL / $\overline{\text{BL}}$): Daten hinein und heraus

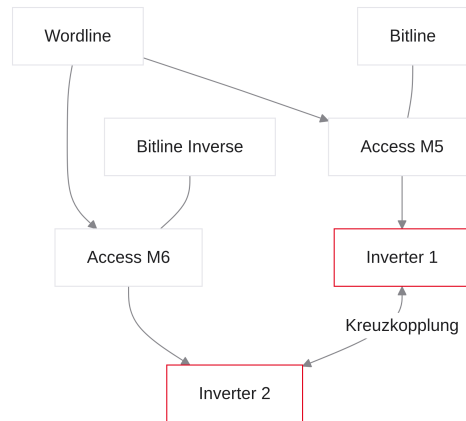
Notizen

Statt rund 20 Transistoren bleiben sechs: zwei Inverter im Ring und zwei Schalter zu den Bitlines. Eine der am häufigsten gebauten Zellen der Welt.

Folie

44 / 88

6T-Zelle: Überblick



Notizen

Wordline steuert die Access-Transistoren; die gekreuzten Inverter halten das Bit. Bitline und inverse Bitline tragen Lesen und Schreiben.

Folie

45/88

Der kreuzgekoppelte Inverter-Ring

- Bit speichern ohne Taktsignal
- Inverter: $1 \rightarrow 0, 0 \rightarrow 1$
- Ausgang von Inverter 1 $\rightarrow$ Eingang von Inverter 2 und zurück
- Stabiler Ring: Zustand hält sich selbst
- Bistabilität: Gleichgewicht, solange Strom anliegt

Notizen

Zwei NOT-Gatter im Kreis: Die linke 1 erzwingt rechts 0, die 0 erzwingt links wieder 1. Das System stabilisiert sich selbst — angetrieben von der Versorgungsspannung.

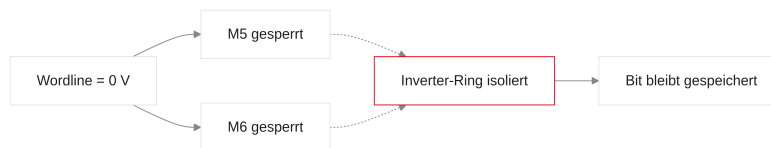
Zustand halten (Hold State)

- Wordline = 0 V: Hold State
- Access-Transistoren sperren
- Zelle von den Bitlines isoliert
- Strom nur im Inverter-Ring
- Ruhestrom theoretisch sehr gering

Notizen

Die meiste Zeit bleibt die Zelle im Hold State: Türen zu, Bit im Ring gefangen. Aktivität auf den Bitlines berührt die Zelle nicht.

Hold State — Schema



Notizen

Schematisch: Gesperrte Access-Transistoren trennen den Ring von den Bitlines.

SRAM Lesen

- Precharge: BL und $\overline{BL}$ auf hohe Spannung
- Wordline High: Access-Transistoren öffnen
- Zelle mit Bitlines verbunden
- Gespeicherte 0 saugt eine Bitline leicht leer
- Sense Amplifier erkennt die Differenz und liefert das Bit

Notizen

Beim Lesen öffnet die Wordline die Türen. Die Null-Seite zieht die Bitline minimal nach unten; Sense Amplifier erkennen die Differenz und melden 0 oder 1 an die ALU.

SRAM Schreiben

- Überwältigung durch stärkere Schreibtreiber
- CPU legt hart 1 auf BL und 0 auf $\overline{BL}$ (oder umgekehrt)
- Wordline High: Türen öffnen
- Schreibtreiber sind größer als die Zell-Inverter
- Inverter kippen — neues Bit ist gefangen

Notizen

Schreiben ist ein physikalischer Kraftakt: Die großen Treiber des Controllers zwingen die winzigen Zell-Inverter um. Nach dem Schließen der Wordline bleibt der neue Zustand im Ring.

Vorteile von SRAM

- Extrem hohe Geschwindigkeit (Sub-Nanosekunden)
- Kein Refresh — keine erzwungenen Pausen
- Einfache Schnittstelle: Adresse anlegen, Daten abholen
- Geringer Stromverbrauch bei Inaktivität
- Integration im selben Logic-Prozess wie die ALU

Notizen

SRAM bremst die Pipeline kaum und lässt sich im gleichen Silizium-Prozess wie die Rechenlogik fertigen — ideal für Caches.

Nachteile von SRAM

- Geringe Datendichte: 6 Transistoren pro Bit brauchen Fläche
- Hohe Kosten: Chipfläche ist teuer
- Kapazitäts-Limit: selten mehr als einige Dutzend MB SRAM als Cache
- Ungeeignet für Massenspeicher (Laptop mit 16 GB SRAM wäre unrealistisch)

Notizen

Sechs Transistoren pro Bit reichen für Caches, nicht für Gigabyte-Hauptspeicher. Deshalb bleibt DRAM unverzichtbar.

Die Leckströme (Leakage)

- Moderne Nodes (z. B. 3 nm): Transistoren sperren nicht mehr perfekt
- Static Leakage auch im Hold State (Quantentunneling)
- Folge: Strom und Wärme nur zum Datenhalten
- Mobile SoCs: Power Gating ganzer SRAM-Blöcke im Standby

Notizen

Bei wenigen Nanometern „lecken“ Transistoren. Große SRAM-Caches können im Standby spürbar Energie ziehen — deshalb Power Gating in Smartphones.

Einsatzgebiet von SRAM im SoC

- Register File: $x0-x31$ im RISC-V-Kern
- L1-Cache: 32–64 KB, extrem niedrige Latenz
- L2-/L3-Cache: geteilt, mehrere MB
- Tightly Coupled Memory (TCM): Echtzeit-Routinen in MCUs

Notizen

SRAM sitzt an der Spitze der Speicherpyramide: Register und Caches. Nur bei Miss geht es weiter zum DRAM.

SRAM in der Hierarchie



Notizen

Schematisch: Jede Stufe fragt die nächste, wenn Daten fehlen. Moderne Kerne sind oft von großen SRAM-Caches umgeben.

Ansicht eines Chip-Dies

- Blick auf ein modernes Silizium-Die unter dem Mikroskop
- Dunkle, regelmäßige Flächen: typischerweise SRAM (L2/L3)
- Unregelmäßige Bereiche: Rechenlogik (ALU, Steuerwerk)
- Moderne Server-/Desktop-CPUs: oft ein großer Anteil der Fläche ist SRAM

Notizen

Bei Die-Shots fallen regelmäßige Gitterstrukturen auf — meist Cache-SRAM. Die eigentliche Logik wirkt unregelmäßiger und belegt oft weniger Fläche als die Caches.

Zusammenfassung SRAM

- Typ: volatiler Arbeitsspeicher
- Zelle: 6T (Inverter-Ring)
- Vorteile: kein Refresh, sehr schnell
- Nachteile: flächenintensiv, Leckströme bei kleinen Nodes
- Einsatz: Caches und CPU-naher Speicher im SoC

Notizen

SRAM ist teuer und schnell — unverzichtbar für die Pipeline. Für den großen Hauptspeicher brauchen wir dichtere Technik: DRAM.

Fragen? — SRAM

- Aufbau der 6T-Zelle?
- Schreib- oder Lesevorgang?
- Warum kein Refresh bei SRAM?

Notizen

Ist das Konzept des selbst-erhaltenden Inverter-Rings klar, bevor wir zu DRAM wechseln?

Volatiler Speicher: DRAM

- Ziel: Gigabyte Arbeitsspeicher zu akzeptablen Kosten
- Weg: Bauteile pro Bit radikal reduzieren
- Ergebnis: Dynamic RAM (DRAM)
- Dynamisch: Zelle hält den Zustand nicht allein — Ladung geht in Millisekunden verloren
- Zelle einfach, Controller komplex

Notizen

Ein 16-GB-Riegel ist DRAM. Maximale Dichte erfordert komplexe Controller, weil die Zellen ihre Ladung nicht dauerhaft halten.

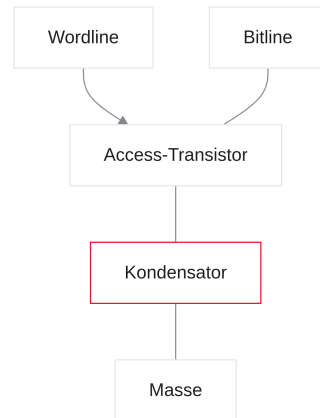
Die 1T1C-Zelle

- Flip-Flop: ≈ 20 Transistoren; SRAM: 6; DRAM: 1 Transistor + 1 Kondensator
- Kondensator: speichert Ladung („Eimer“)
- Transistor: Schalter zum Füllen/Leeren
- Weniger Bauteile pro Bit sind kaum möglich

Notizen

1T1C ist der Dichte-Kompromiss: Ein Transistor, ein Kondensator. Maximale Packungsdichte — um den Preis von Refresh und komplexer Ansteuerung.

1T1C — Schema



Notizen

Wordline öffnet den Transistor; der Kondensator speichert die Ladung gegen Masse.

Folie

61 / 88

Kondensator als Speicher

- Geladen (hohe Spannung) → logische 1
- Entladen ($\approx 0\text{ V}$) → logische 0
- Kapazitäten im Femtofarad-Bereich — einige zehntausend bis hunderttausend Elektronen
- 3D-Fertigung: Trench-/Stack-Kondensatoren für Dichte

Notizen

Um Kapazität bei kleinem Fußabdruck zu gewinnen, werden Kondensatoren als Gräben oder Stapelstrukturen gefertigt — hochpräzise Halbleitertechnik.

Folie

62 / 88

Was bedeutet „dynamisch“?

- Isolation ist nie perfekt — Ladung sickert ab
- Spannung einer „1“ sinkt kontinuierlich
- Nach ca. 64 ms oft kritisch für die Sense Amplifiers / Leseverstärker
- Ohne Pflege löscht sich DRAM selbst

Notizen

Das „D“ steht für dynamisch: Die Zelle vergisst ohne periodisches Auffrischen. Metapher: Eimer mit kleinem

Folie

63 / 88

Der Refresh-Zyklus

- Memory Controller muss Zellen regelmäßig auffrischen
- Typisch: innerhalb von ≈ 64 ms jede Zelle lesen und zurückschreiben
- Während Refresh: RAM oft blockiert — CPU kann warten müssen
- Energie: Refresh kostet Strom auch im Standby

Notizen

Bevor die Ladung kritisch wird, liest und schreibt der Controller zurück. Das passiert dauernd im Hintergrund — deshalb verbraucht DRAM auch ohne „sichtbare“ Arbeit Energie.

Folie

64 / 88

Refresh — Schema



Notizen

Schematisch: Ladung sinkt, Refresh stellt den Pegel wieder her. Zeiten sind typische Größenordnungen, keine Messdaten.

Folie

65 / 88

DRAM-Lesen ist destruktiv

- Lesen: restliche Ladung fließt auf die Bitline
- Spannungsausgleich: Kondensator danach weitgehend leer
- Destructive Read: die Information wird durchs Messen zerstört
- Pflicht: Wert sofort zurückschreiben

Notizen

Wie ein Tropfen in einem großen Rohr: Nach dem Messen ist der Tropfen „zerlaufen“. Hardware muss die Zelle neu füllen.

Sense Amplifiers

- Leseverstärker am Rand der Matrix
- Vergleichen winzige Bitline-Schwankung mit Referenz
- Kippen auf volle digitale Pegel (0 / 1)
- Nebeneffekt: oft automatisches Zurückschreiben der Ladung

Notizen

Ohne Sense Amplifier wäre DRAM unbrauchbar. Sie verstärken Millivolt-Differenzen und stellen dabei typischerweise die Zellenladung wieder her.

DRAM Schreiben

- Bitline hart auf High oder Low treiben
- Wordline öffnet den Access-Transistor
- Kondensator füllt/entleert sich schnell
- Wordline zu — Bit gefangen (bis zum nächsten Refresh)

Notizen

Schreiben ist der einfachere Teil: Treiber setzen den Pegel, Transistor verbindet, Kondensator übernimmt.

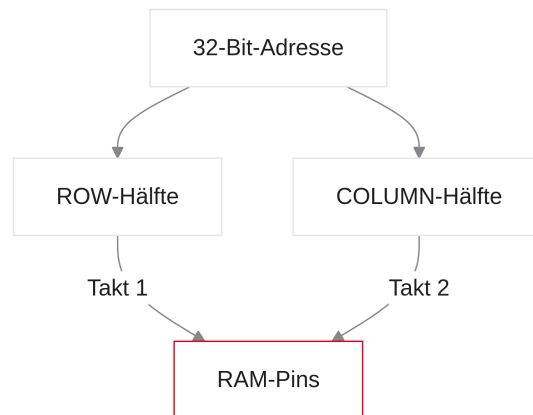
Adress-Matrix und Multiplexing

- Milliarden Adressen — nicht Milliarden Pins
- Zellen in Rows und Columns (2D-Gitter)
- 32-Bit-Adresse wird geteilt: erst Row, dann Column über dieselben Pins

Notizen

Multiplexing spart Gehäuse-Pins: Zuerst Zeile, dann Spalte — nacheinander über denselben Adressbus.

Multiplexing — Schema



Notizen

Schematische Aufteilung; konkrete Bitbreiten hängen vom Modul ab.

RAS und CAS

- RAS (Row Address Strobe): „jetzt kommt die Zeile“
- CAS (Column Address Strobe): „jetzt kommt die Spalte“
- Nacheinander senden kostet Takte — Teil der DRAM-Latenz

Notizen

RAS/CAS kennen Sie ggf. aus BIOS-Timings. Niedrigere CAS-Latenz bedeutet schnelleres Auffinden der Spalte — bei gleichen übrigen Randbedingungen.

Warum DRAM langsamer ist als SRAM

- Adress-Multiplexing (Row, dann Column)
- Analoges Auslesen und Sense Amplifier
- Destruktives Lesen inkl. Zurückschreiben / Precharge
- Mögliche Wartezeiten während Refresh

Notizen

Die Latenz folgt aus Physik und Organisation — nicht aus „Böswilligkeit“. Der Inverter-Ring des SRAM antwortet deutlich direkter.

SDRAM und DDR

- SDRAM: Taktsignal, synchrone Zugriffe, Pipelining möglich
- DDR: Daten bei steigender und fallender Flanke
- Bandbreite steigt bei gleichem Takt spürbar
- LPDDR: energieoptimierte Varianten für Mobile

Notizen

DDR4/DDR5 bleiben 1T1C-Zellen im Kern. „Double Data Rate“ betrifft die Schnittstelle, nicht die Zellphysik.

Einsatzgebiet von DRAM im SoC

- Meist nicht auf dem Logic-Die (anderer Prozess)
- Separater Chip oder Package-on-Package
- Main Memory: OS, Heap (`malloc`), große Arrays
- Kleine MCUs: oft nur interner SRAM, kein DRAM

Notizen

Große SoCs (z. B. Linux-fähig) brauchen DRAM. Kleine Sensor-MCUs kommen oft mit Kilobyte-SRAM aus.

Vergleich: SRAM vs. DRAM

	SRAM	DRAM
Zelle	6T Inverter-Ring	1T1C
Datenerhalt	stabil (mit Strom)	Refresh nötig
Geschwindigkeit	sehr hoch	mittel
Dichte	gering	sehr hoch
Einsatz	Caches, Register	Hauptspeicher
Kosten / Bit	hoch	niedrig

Notizen

Kein „besser“ oder „schlechter“ — beide sind für ihre Ebene in der Hierarchie optimiert. SRAM ist der schnelle Kurier, DRAM das Containerschiff.

Spickzettel SRAM / DRAM

Eigenschaft	SRAM	DRAM
Zellen-Architektur	6 Transistoren	1T + 1C
Datenerhalt	stabil (Strom an)	flüchtig (Refresh)
Geschwindigkeit	$< 1 \text{ ns}$ (typ.)	$\approx 50\text{--}100 \text{ ns}$ (typ.)
Speicherdichte	gering	extrem hoch
Einsatz im SoC	L1/L2/L3, Register	Main Memory
Kosten pro Bit	hoch	niedrig

Angaben schematisch — typische Größenordnungen.

Notizen

Diese Dimensionen reichen oft für Klausur und Interview: Cache vs. Hauptspeicher physikalisch begründet.

Fragen? — DRAM

- Kondensator-Prinzip?
- Destruktives Lesen?
- Refresh-Zyklus klar?

Notizen

Ist verständlich, warum Refresh und Multiplexing die Pipeline verzögern können?

Spezialspeicher im SoC

- Neben Cache und Main Memory: spezialisierte RAM-Zellen
- Problem: CPU und Peripherie mit unterschiedlichen Takten
- Wie Daten austauschen ohne gegenseitiges Blockieren?
- Lösungen: Dual-Ported RAM (DPRAM) und FIFOs

Notizen

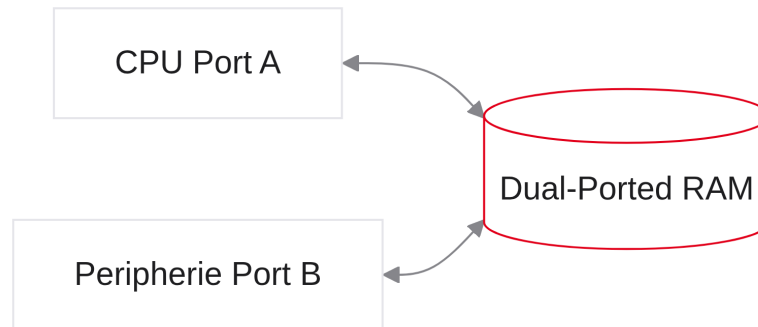
Manchmal müssen zwei Module gleichzeitig auf denselben Puffer zugreifen. Normales Single-Port-RAM reicht dann nicht.

Dual-Ported RAM (DPRAM)

- SRAM mit zwei unabhängigen Ports
- Eigene Adress-/Daten-/Steuerleitungen pro Port
- Gleichzeitig lesen/schreiben an verschiedenen Adressen möglich
- Physik oft 8T (zusätzliche Access-Transistoren)
- Einsatz: Shared Memory, Videopuffer, ...

Notizen

Port A und Port B können parallel arbeiten — teurer in Fläche, aber ohne Bus-Streit bei getrennten Adressen.

DPRAM — Schema**Notizen**

Zwei Ports, ein Speicherblock — Grundlage für nebenläufige Produzenten/Konsumenten auf dem Chip.

Kollisionsvermeidung im DPRAM

- Gleichzeitiges Schreiben derselben Zelle: Ergebnis undefiniert
- Arbitration Logic als Schiedsrichter
- Busy-Flag: zweiter Zugriff verzögert / Stall

Notizen

Bei Adresskonflikt entscheidet Hardware, wer wartet — sonst Metastabilität.

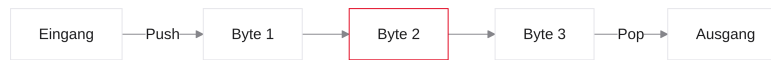
FIFO-Puffer

- First-In-First-Out: Warteschlange
- Keine expliziten Adressen für Push/Pop
- Hardware: SRAM mit Write-/Read-Pointer (Ringpuffer)

Notizen

FIFO ist die Standard-Schnittstelle zwischen unterschiedlichen Taktraten und Datenraten.

FIFO — Schema



Notizen

Daten werden eingeschoben und in derselben Reihenfolge entnommen — ohne dass Software jede Adresse verwaltet.

Hardware-FIFOs für I/O

- Beispiel UART: Bytes kommen langsam an
- Ohne FIFO: Interrupt pro Byte/Bit — hohe CPU-Last
- Mit FIFO: füllen bis Watermark, dann CPU wecken
- Grundlage für effiziente Bus-Peripherie (später Block 5)

Notizen

FIFOs entkoppeln schnelle CPU und langsame I/O. In späteren Blöcken nutzen wir das bei seriellen Schnittstellen.

Brückenschlag zur Software

- CPU sieht keinen Kondensator — nur linearen 32-Bit-Adressraum
- Adress-Decoder lenkt Bereiche an physische Bausteine
- Beispiel: Pointer auf Flash vs. Pointer auf SRAM/DRAM

Notizen

Memory-Mapping verbindet C-Pointer mit der Physik. Der Compiler kennt Transistoren nicht — der Decoder schon.

Memory Map eines Mikrocontrollers

Typische Aufteilung (Beispiel, schematisch):

- 0x0000_0000: Boot-ROM
- 0x0800_0000: Flash (Programm)
- 0x2000_0000: SRAM (Variablen)
- 0x4000_0000: Peripherie / MMIO

Ohne Datenblatt-Memory-Map kein zuverlässiges Booten.

Notizen

Falsche Adressen führen zu Bus Faults. Die Memory Map ist Pflichtlektüre jedes MCU-Projekts.

Das Linker-Skript

- .ld-Datei beschreibt die Memory Map für den Linker
- .text → Flash; .data / .bss → SRAM
- Ohne Skript keine korrekte Firmware-Platzierung

```
/* Beispiel eines simplen Linker-Skripts (Ausschnitt) */  
MEMORY {  
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 256K  
    RAM (rwx)  : ORIGIN = 0x20000000, LENGTH = 64K  
}
```

Notizen

Das Linker-Skript weist Abschnitte den physischen Regionen zu. So landet Code im Flash und Variablen im SRAM.

Ausblick Labor (Renode)

Geplantes Laborziel (Details folgen mit dem Übungsblatt):

- C-Projekt für einen RISC-V-SoC kompilieren
- Linker-Skript und Memory Map untersuchen
- Binary in Renode laden
- Speicherinhalte beobachten

Notizen

Sobald das Übungsblatt steht, verifizieren Sie die Konzepte im Simulator — nicht nur „Kompilieren und hoffen“.

Zusammenfassung Block 1

- Speicherphysik: Kompromisse aus Kosten, Geschwindigkeit, Dichte
- Flash (non-volatil): Firmware; NOR oft mit XIP in MCUs
- SRAM (6T): schnell, teuer — Caches und Register
- DRAM (1T1C): dicht, Refresh — Hauptspeicher
- Linker-Skript verteilt Software auf physische Zonen

Notizen

Damit ist Block 1 geschlossen. In Block 2 (Vorlesung 2) sehen wir, wie Caches die DRAM-Latenz verbergen. Vielen Dank für Ihre Aufmerksamkeit.